



PANDA LAND SAFE BOX

Cálculo do número mínimo de rolagens para abrir o cofre

Problema p1235 · Trabalho de Algoritmos e Estruturas de Dados

Grafos

Árvore Geradora Mínima

Algoritmo de Prim

Python puro

O Problema

Um cadeado de 4 dígitos, várias chaves, e o desafio de abrir tudo gastando o menos possível.



Cadeado de 4 dígitos

Cada dígito vai de 0 a 9 e é circular: depois do 9 volta ao 0, e antes do 0 vai ao 9. Começa em 0000.



Várias chaves (N)

O cofre só abre quando TODAS as N chaves são destravadas. Para destravar, gira-se até a chave e aperta UNLOCK.



Botão JUMP (grátis)

Teleporta os dígitos para qualquer chave JÁ destravada, sem custo nenhum de rolagem.



Mínimo de rolagens

Cada giro de um dígito custa 1. O cofre só abre se o total for o mínimo possível — senão, reseta.

Medindo a Distância

Quantas rolagens custa transformar uma combinação em outra?

Cada dígito é circular

O custo entre dois dígitos a e b é o menor entre subir ou descer:

$$\min(|a - b|, 10 - |a - b|)$$

A distância total entre duas chaves é a **soma dos 4 dígitos**.

Ex.: de 0 até 9 custa só 1 (desce uma posição), não 9.

Exemplo: $\text{dist}(0000, 9113)$

Dígito 1	0 → 9	$\min(9, 1) = 1$
Dígito 2	0 → 1	$\min(1, 9) = 1$
Dígito 3	0 → 1	$\min(1, 9) = 1$
Dígito 4	0 → 3	$\min(3, 7) = 3$
Total = 1 + 1 + 1 + 3 = 6 rolagens		



A Sacada: é um problema de Grafos

Com o botão JUMP, abrir o cofre vira um problema clássico: ligar tudo gastando o mínimo.

Nós

Cada chave é um nó do grafo (mais o ponto inicial 0000).

Arestas

Entre todo par de nós, com peso = distância de rolagem.

JUMP = grátis

Cada chave nova só paga o caminho até a chave aberta mais próxima.

Conectar todas as chaves com o menor custo total = Árvore Geradora Mínima (MST)

Detalhe do 0000: ele não é uma chave. Sai-se dele uma vez e nunca se volta via JUMP. Entra como **folha** da árvore.

Resposta = MST entre as chaves + menor distância de 0000 até alguma chave

Algoritmo de Prim

A ideia é simples: a cada passo, conecte o ponto mais barato que ainda falta — e repita até montar a árvore inteira.

mst_cost.py

```
def mst_cost(keys):
    n = len(keys)
    if n <= 1: return 0
    in_tree = [False] * n
    min_edge = [INF] * n
    min_edge[0] = 0
    total = 0
    for _ in range(n):
        # 1) ESCOLHER o no mais barato
        u, best = -1, INF
        for v in range(n):
            if not in_tree[v] and min_edge[v] < best:
                best, u = min_edge[v], v
        # 2) ADICIONAR esse no a arvore
        in_tree[u] = True
        total += best
        # 3) ATUALIZAR os vizinhos
        for v in range(n):
            if not in_tree[v]:
                d = dist(keys[u], keys[v])
                if d < min_edge[v]:
                    min_edge[v] = d
    return total
```

Como ele monta a árvore

A cada volta, o programa pega a chave mais barata de conectar, junta ela à árvore somando o custo, e recalcula se as outras ficaram mais perto. Repete até todas entrarem.



E a fila de prioridade?

Esta é a versão do Prim com lista simples: para achar a chave mais barata, ela percorre todas a cada passo. Isso custa $O(N^2)$.

A fila de prioridade (heap) só compensa quando o grafo tem poucas conexões. Aqui o grafo é completo — toda chave se liga a todas — então a versão simples é a mais indicada.

Por que funciona e quão rápido é



A solução é ótima

O Prim é "guloso": a cada passo, pega sempre a ligação mais barata na hora.

Parece arriscado, mas um teorema (o Teorema do Corte) garante que essa escolha sempre leva ao melhor resultado no final.

Ou seja: não existe um jeito de abrir o cofre gastando menos que o nosso resultado.

Tentar todas as ordens possíveis seria inviável ($O(N!)$). É a MST que torna o problema rápido.



Complexidade

Algoritmo de Prim

$O(N^2)$

Distância do 0000

$O(N)$

Por caso de teste

$O(N^2)$

Total (T casos)

$O(T \cdot N^2)$

Pior caso ($N=500$): ~250 mil operações por caso. Rápido o bastante para qualquer limite de tempo.



Casos Especiais

O ponto 0000

Não é chave: entra como folha, ligado a uma única chave. Por isso é somado à parte.

Uma só chave (N=1)

Não há árvore (MST = 0). A resposta é só a distância do 0000 até ela.

Chaves repetidas

Distância zero entre elas: a aresta é absorvida sem custo (um JUMP grátis).

Dígitos circulares

0 e 9 são vizinhos. $\min(|a-b|, 10-|a-b|)$ evita superestimar as rolagens.

Leitura da entrada

Lê tudo em tokens — funciona com chaves em uma linha ou em várias.

Zeros à esquerda

`zfill(4)` mantém '0213' como '0213' e não como '213'.



Resultados

Testado com os 4 casos do enunciado — todos batem com o esperado.

Entrada (entrada.txt)

```
4
2 1155 2211
3 1111 1155 5511
3 1234 5678 9090
4 2145 0213 9113 8113
```

Saída obtida = esperada

16

20

26

17



4 de 4 casos corretos

Comando: `python p1235.py < entrada.txt`

Exemplo Passo a Passo

Chaves: 1111, 1155, 5511 — veja como o JUMP economiza rolagens.

0000 → 1111

Rola do início até a 1ª chave

+4

1111 → 1155

Rola para a 2ª chave

+8

JUMP → 1111

Volta de graça (já destravada)

grátis

1111 → 5511

Rola para a 3ª chave

+8

Total = 4 + 8 + 8 = 20 rolagens (o JUMP não conta!)



Em resumo

Modelado como grafo

Resolvido como MST

Prim do zero

0000 como caso especial

Resultado correto em todos os testes, em Python puro, sem bibliotecas externas.

Obrigado! • Perguntas?